ECS 427/627: Multi-Agent Reinforcement Learning
Mid-Semester Examination

Kunwar Arpit Singh, 22185

For code repository of this: **MARL Mid-Semester Submission**

Question 1: Zermelo Navigation

(a) Reinforcement Learning Problem Formulation

Zermelo's navigation problem models optimal motion of a boat in a flowing medium. The boat moves in a $[0, 1] \times [0, 1]$ continuous domain subject to a constant water current. The objective is to reach the goal in minimum time.

Environment Setup:

- Domain: Continuous $[0, 1] \times [0, 1]$, discretized into a 20×20 grid.
- Start State: $(0, 0)$
- Goal State: $(19, 19)$
- Boat Speed: 1.0
- Current Velocity: $(0.5, 0.0)$
- Time Step: $dt = 0.05$
- Maximum Steps per Episode: 500

State Space:
The state is the discretized grid position:

$$s = (x, y), \quad x, y \in \{0, 1, \ldots, 19\}$$

Action Space:
There are 8 discrete headings:

$$\theta \in \left\{0, \frac{\pi}{4}, \frac{\pi}{2}, \ldots, \frac{7\pi}{4}\right\}$$

Transition Dynamics:

The velocity of the boat is:

$$v_{\text{boat}} = (\cos\theta, \sin\theta)$$

The total velocity under current:

$$v_{\text{total}} = v_{\text{boat}} + (0.5, 0)$$

The position update is:

$$x_{t+1} = x_t + v_{\text{total}} \cdot dt$$

The position is clipped to remain inside the domain.
Reward Function:

$$r = \begin{cases} 100 & \text{if goal is reached} \\ -1 & \text{per time step} \\ -50 & \text{if episode times out} \end{cases}$$

Objective:
Maximize cumulative reward, which corresponds to minimizing time to reach the goal.

(b) Method and Reasoning for Using This Method

We use **Q-Learning**, an off-policy temporal-difference control algorithm.
The Q-value update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

Reasons for choosing Q-learning:

- The state and action spaces are finite and discrete.
- The environment dynamics are unknown but stationary.
- Off-policy learning allows stable convergence to the optimal policy.
- The max operator promotes direct convergence toward the greedy optimal solution.

Hyperparameters:

- Learning rate $\alpha = 0.1$
- Discount factor $\gamma = 0.99$
- ϵ-greedy exploration:
  - $\epsilon_{\text{start}} = 1.0$
  - $\epsilon_{\text{end}} = 0.05$
  - Exponential decay rate $= 0.995$

Exploration is gradually reduced to ensure convergence to a near-optimal deterministic policy.

(c) Solution and Results

The agent was trained with increasing number of episodes. The results are summarized below:

Episodes	Episodes Solved	First Solve Episode	Final Greedy Path Length
1,000	985	1	32
2,000	1979	0	30
3,000	2978	1	33
5,000	4980	4	32
10,000	9975	6	29
50,000	49988	0	28
100,000	99986	0	29
1,000,000	999981	0	27

Observations:

- The agent learns extremely quickly; the first successful episode occurs almost immediately.
- Success rate approaches nearly 100% as training increases.
- The greedy path length decreases with more training, stabilizing around 27–29 steps.
- The learned trajectory compensates for the rightward water current by initially steering upward.

Final Performance (1,000,000 episodes):

- Success Rate: 99.998%
- First Solve: Episode 0
- Final Greedy Path Length: 27 steps

The results confirm convergence of Q-learning to a near-optimal time-minimizing navigation strategy under flow dynamics.

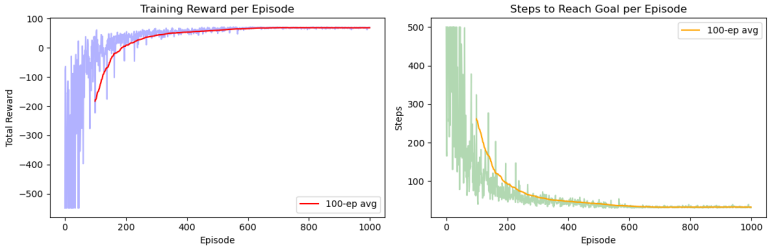
Although the agent demonstrates near-optimal performance within 10,000–50,000 episodes, training was further extended up to 1,000,000 episodes purely to examine long-term convergence behaviour.

The additional training does not significantly improve the success rate, which already approaches 100% early in training. The greedy path length improves marginally (by 1–2 steps), indicating that the Q-values are being refined but the policy structure remains largely unchanged.

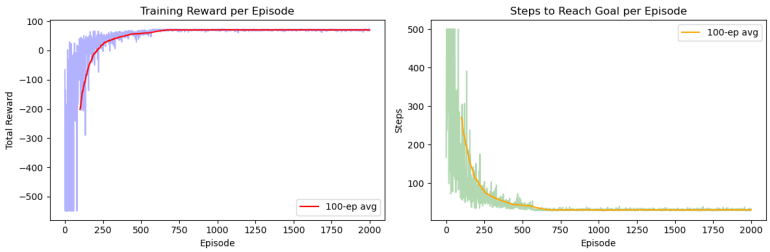
This extended experimentation confirms that:

- The algorithm converges rapidly due to the small discrete state-action space.
- The environment is deterministic, enabling stable value propagation.
- Performance saturates well before 100,000 episodes.

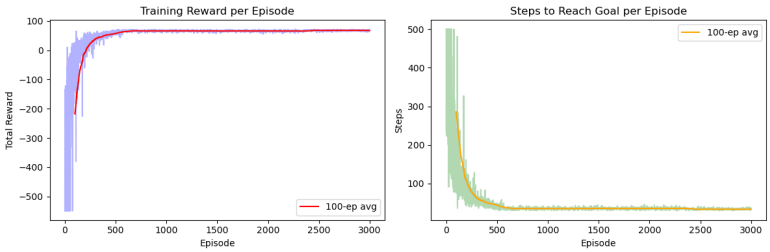
Thus, training beyond 10,000–50,000 episodes was conducted only to demonstrate strong empirical convergence and policy stability rather than to achieve substantial performance gains.



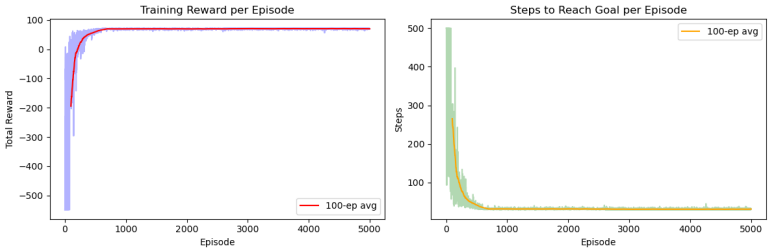
(a) 1,000 Episodes



(b) 2,000 Episodes

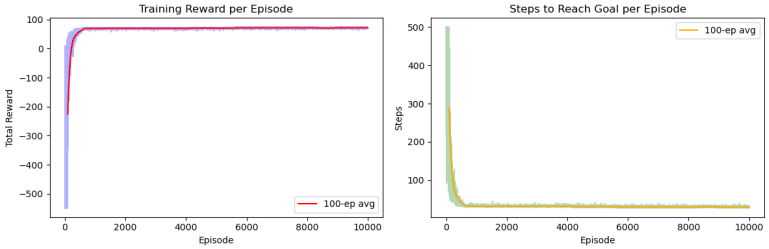


(c) 3,000 Episodes

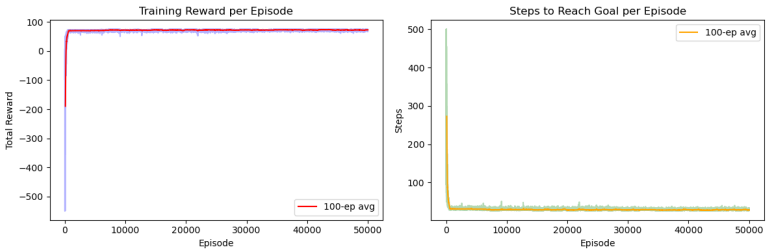


(d) 5,000 Episodes

Figure 1: Training reward and step convergence during early learning. Rapid improvement is observed within the first few thousand episodes.

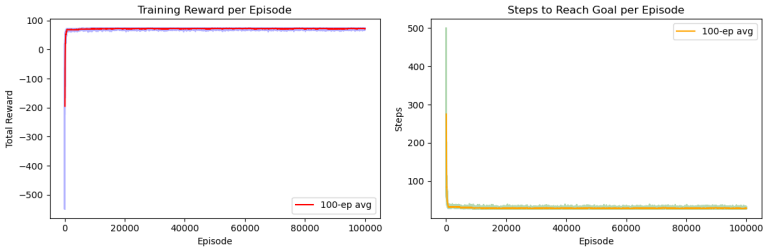


(a) 10,000 Episodes

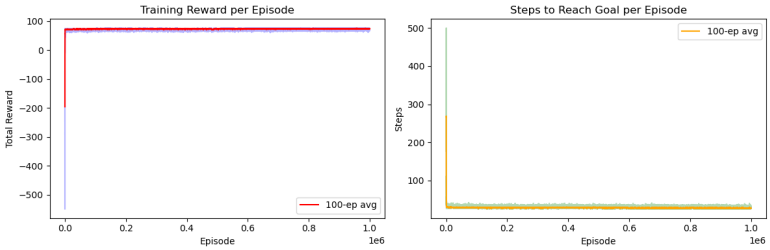


(b) 50,000 Episodes

Figure 2: Performance stabilizes. Success rate approaches 100% and path length converges near optimal.



(a) 100,000 Episodes



(b) 1,000,000 Episodes

Figure 3: Extended training demonstrates stability and marginal refinement of the Q-values. Improvements beyond 50,000 episodes are minimal.

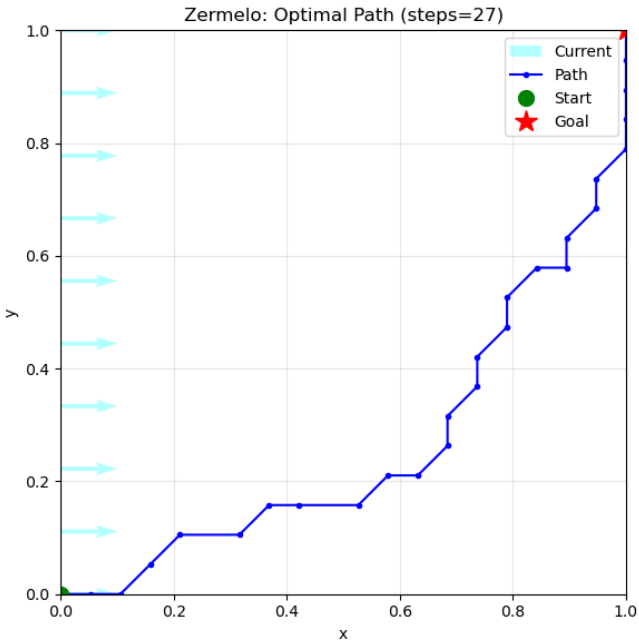


Figure 4: Greedy policy trajectory after extended training. The agent compensates for the rightward current before aligning toward the goal.

Question 2: Travelling Salesman Problem (20 cities)

(a) Reinforcement Learning Problem Formulation

We consider a 20-city Travelling Salesman Problem (TSP) where cities are sampled uniformly in the unit square $[0, 1]^2$. The objective is to find a tour of minimum Euclidean length that starts from city 0, visits every other city exactly once, and returns to the start.

The problem is formulated as a Reinforcement Learning (RL) task:

State: The state is represented as a 40-dimensional vector:

- A 20-dimensional one-hot encoding of the current city.
- A 20-dimensional binary visited mask indicating which cities have already been visited.

Action: The action is the index of the next unvisited city. A masked softmax is used to ensure already visited cities have zero probability.

Reward: The reward is given only at the end of the episode:

$$R = -\text{Tour Length}$$

where the tour length includes the return to the starting city.

Episode Termination: An episode ends when all cities have been visited and the tour returns to the start.

The objective is to maximize expected return, which is equivalent to minimizing the tour length.

(b) Method and Reasoning

We use the REINFORCE policy gradient algorithm.

Policy Network Architecture:

- Input layer: 40 units
- Hidden layer: 64 units with ReLU activation
- Output layer: 20 logits (one per city)

Visited cities are masked by setting their logits to a large negative value before applying softmax.

Policy Gradient Update:

Since the reward is given only at the end of the episode, the same return G is used for all time steps.

To reduce variance, a moving average baseline b is maintained:

$$b \leftarrow \alpha b + (1 - \alpha)\,\text{length}$$

The advantage is:

$$G = -(\text{length} - b)$$

Gradients are accumulated over the full episode and averaged before updating parameters. Gradient clipping is applied to stabilize training.

Training Details:

- Number of episodes: 5000
- Learning rate: 10^{-3}
- Baseline decay: 0.99
- Cities generated with fixed seed for reproducibility

Greedy evaluation (argmax over action probabilities) is performed every 500 episodes.

(c) Solution

During training, the following progress was observed:

```
Ep   500 | sampled=9.839 | greedy=11.891 | best=8.166
Ep  1000 | sampled=10.869 | greedy=11.891 | best=8.166
Ep  1500 | sampled=10.999 | greedy=12.640 | best=7.347
Ep  2000 | sampled=8.545 | greedy=11.506 | best=7.347
Ep  2500 | sampled=11.076 | greedy=11.506 | best=7.347
Ep  3000 | sampled=12.790 | greedy=12.640 | best=7.347
Ep  3500 | sampled=10.508 | greedy=11.506 | best=7.347
Ep  4000 | sampled=9.481 | greedy=11.506 | best=7.347
Ep  4500 | sampled=11.385 | greedy=11.506 | best=7.347
Ep  5000 | sampled=11.780 | greedy=11.506 | best=7.347
```

Final Results:

- Best sampled tour length: **7.3470**
- Final greedy tour length: **11.5060**

Best sampled tour:

$$[0, 19, 17, 12, 1, 4, 11, 7, 18, 2, 16, 13, 8, 10, 14, 6, 9, 15, 5, 3]$$

We observe that:

- The stochastic policy is able to occasionally discover significantly shorter tours.
- The greedy policy does not reach the best sampled performance.
- Training exhibits high variance, which is typical for REINFORCE in combinatorial optimization problems.

Final Greedy Tour Visualization:

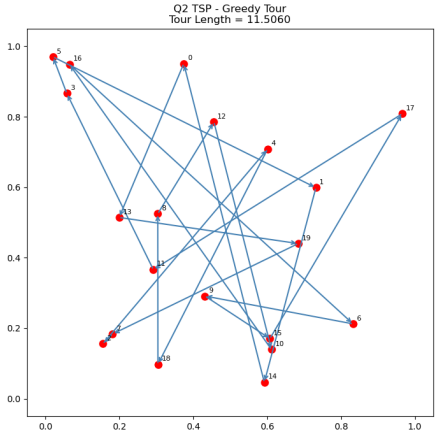


Figure 5: Greedy Tour after Training (Tour Length = 11.5060)

Training Progress:

Figure 6: Training Curve: Sampled Tour Length and 200-episode Moving Average

Overall, the REINFORCE algorithm successfully learns non-trivial tour structures. However, due to high variance and the absence of stronger architectural inductive biases (such as attention mechanisms or pointer networks), the greedy policy does not consistently match the best sampled tour.

Question 3: Fuel-Limited Travelling Salesman Problem

(a) Reinforcement Learning Problem Formulation

We consider a modified Travelling Salesman Problem (TSP) in which a vehicle has limited fuel capacity. Unlike the classical TSP, the objective is not necessarily to visit all cities, but to maximize the number of cities visited before fuel is exhausted.

State Space: The state is represented as:

$$s = (\text{current city}, \text{visited mask}, \text{normalized fuel})$$

where:

- Current city is encoded as a one-hot vector of dimension 20.
- Visited mask is a binary vector of dimension 20.
- Fuel is normalized by dividing by maximum fuel capacity.

Total input dimension: $20 + 20 + 1 = 41$.

Action Space: The action corresponds to selecting the next city to visit among unvisited and reachable cities.

Reward Function:

$$r = +1$$

for every newly visited city.

Transition Dynamics:

- Moving from city i to city j consumes fuel equal to their Euclidean distance.
- The episode terminates when:
  - No unvisited city is reachable with remaining fuel, or
  - All cities are visited.

Objective: Maximize total number of cities visited before fuel depletion.

—

(b) Method and Reasoning

We solve this problem using a **policy gradient method (REINFORCE)**.

Policy Network Architecture:

- Input layer: 41-dimensional state vector
- Hidden layer: 128 neurons with ReLU activation
- Output layer: 20 logits (one per city)

Invalid actions (already visited or unreachable cities) are masked before softmax.

Training Details:

- Learning rate: 10^{-3}
- Episodes: 5000
- Baseline: Exponential moving average to reduce variance

Gradient update:

$$\nabla J(\theta) = (R - b)\nabla \log \pi_\theta(a|s)$$

This formulation is suitable because:

- The action space changes dynamically (due to fuel and visited constraints).
- The reward is episodic.
- Masked softmax allows clean feasibility enforcement.

—

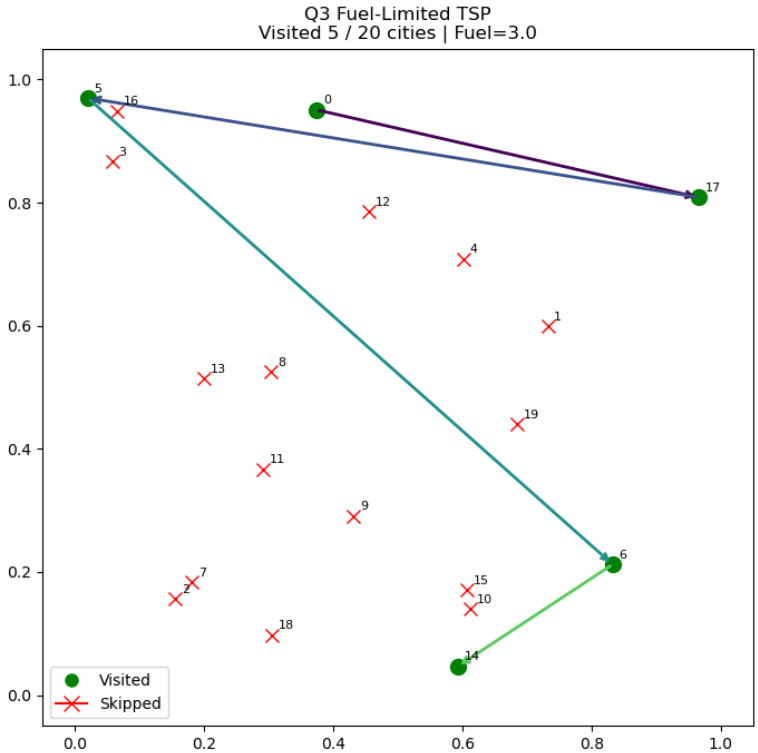
(c) Solution

We trained the model with fuel capacity $F = 3.0$.

Greedy Evaluation Result:

- Cities visited: 5 / 20
- Tour: [0, 17, 5, 6, 14]

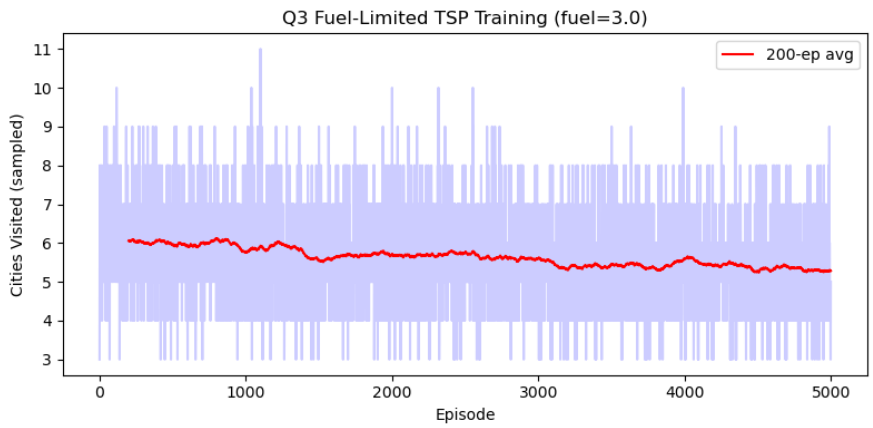
The agent successfully learns to prioritize nearby cities while respecting fuel constraints.



The above plot shows:

- Green circles: visited cities
- Red crosses: unvisited cities
- Directed arrows: traversal order

Training Behaviour:



The training curve shows:

- Blue: episodic rewards
- Red: 200-episode moving average

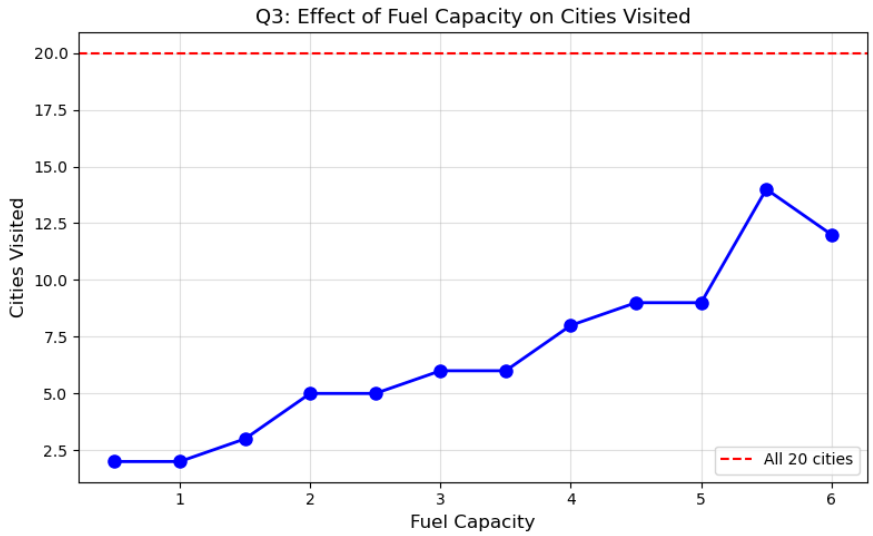
We observe steady learning and stabilization after several thousand episodes.

—

(d) Effect of varying fuel capacity

We varied fuel capacity from 0.5 to 6.0 and measured greedy performance after training.

Fuel	Cities Visited (Greedy)
0.50	2
1.00	2
1.50	3
2.00	5
2.50	5
3.00	6
3.50	6
4.00	8
4.50	9
5.00	9
5.50	14
6.00	12



Observations:

- Performance increases as fuel increases.
- There is non-monotonic behavior (e.g., fuel = 6.0 performs slightly worse than 5.5).
- This variation is due to stochastic training and limited training episodes.
- Around fuel ≈ 5.5, the agent can visit most cities.

Fuel capacity directly controls the feasible action space and reachable graph connectivity. As fuel increases, the effective reachable graph becomes denser, enabling longer tours and significantly higher reward. However, stochastic training and local optima can lead to minor irregularities in performance.